Requisitos de la práctica, punto a punto

AGR — Práctica 2: qué pide el enunciado, cómo se ha implementado y cómo se comprueba

Pablo Elvira Calatayud

Índice

- Cómo está organizado
- PARTE 1 — Virtualización de servicios web con Vagrant (págs. 1–2)
  - R1.1. Script Vagrant que despliega una VM Linux
  - R1.2. Servidor Node.js accesible desde la VM y desde el exterior
  - R1.3. Instalar Node, descargar el servidor de GitHub y ejecutarlo
  - R1.4. Usar SSR-master-server (o uno propio)
  - R1.5. Redirección de puertos 8080 → 3000
  - R1.6. Dirección de red acorde al tipo de conexión
  - R1.7. Entregar el script Ruby
  - R1.8. Configuraciones adicionales y esquemas de red avanzados
  - R1.9. Estudio de consumo de memoria y CPU
  - R1.10. Consumo de una red que replique el laboratorio
  - R1.11. Vigilar con `watch` cómo crecen las imágenes
  - R1.12. Comparar con servidores comerciales
  - R1.13. Documento opcional de pruebas y configuraciones
- PARTE 2 — Virtualización de infraestructura de red con KVM (págs. 3–5)
  - R2.1. Script que despliegue la red del laboratorio 3103 con QEMU-KVM
  - R2.2. Lenguaje del script
  - R2.3. Generar los ficheros XML de todas las VMs
  - R2.4. Uso de una librería XML (lxml)
  - R2.5. Las cuatro subredes inferiores pueden ser un único PC
  - R2.6. Desplegar todas las máquinas
  - R2.7. Instalar la *Software Routing Suite* en los routers
  - R2.8. Rutas estáticas para conectividad global
  - R2.9. Servicio web en el servidor
  - R2.10. Acceso desde la parte inferior al servicio web
  - R2.11. Acceso del anfitrión al servidor
  - R2.12. Entregar el script
  - R2.13. Memoria: plan de numeración con dibujo
  - R2.14. Memoria: tablas de enrutamiento de los routers
  - R2.15. Plan de pruebas que confirme la conectividad global

    - R2.16. Configuraciones adicionales en las VMs y en la Software Routing Suite
    - R2.17. Usar la imagen de disco y el XML de Moodle
    - R2.18. Usuario root / agr; cambiar hostname y contraseña
    - R2.19. Conectar el anfitrión a un bridge de las subredes inferiores (aconsejado)
    - R2.20. Mensajes con print y trazas con logging
    - R2.21. Gestionar las VMs con `virsh define|start|shutdown|destroy`
    - R2.22. Terminales con `xterm -e "sudo virsh console …" &`
    - R2.23. `virt-cat` / `virt-ls` para inspeccionar sin entrar
- Resumen

Cómo está organizado

Para **cada requisito** del enunciado (`doc/development/enunciado/Practica_2_enunciado.pdf`) se indica:

- **Enunciado:** la cita literal y la página.
- **Qué hay que implementar:** la interpretación técnica del requisito.
- **Cómo se ha implementado:** la solución, con las órdenes o el código clave.
- **Dónde está:** los ficheros del repositorio.
- **Cómo se comprueba:** la prueba y su resultado.
- **Estado:** ✅ cumplido · ⚠️ cumplido con matices o pendiente de confirmar · ❌ no cumplido.

La parte 2 está implementada **a mano** en `parte2/`: plantilla de Moodle, XML de las 9 VMs, `red.sh` (prepara el anfitrión), preparación del molde y configuración interna de cada VM. El proceso completo está en `guia_paso_a_paso.pdf`. **Queda pendiente el script que automatice todo el despliegue**; los requisitos que dependen de él se marcan así.

(Una primera versión automatizada con otro diseño se conserva como material de trabajo en `doc/development/historico_script/`; no forma parte de la entrega.)

PARTE 1 — Virtualización de servicios web con Vagrant (págs. 1–2)

R1.1. Script Vagrant que despliega una VM Linux

Enunciado (pág. 1): «…pedimos crear un script Vagrant (Ruby) para desplegar una máquina virtual Linux (se podrá emplear la distribución que se prefiera)…»

Qué hay que implementar: un `Vagrantfile` que, con `vagrant up`, cree y configure la VM sin pasos manuales.

Cómo se ha implementado:

- **Box:** `bento/ubuntu-22.04` con proveedor VirtualBox (forzado con `VAGRANT_DEFAULT_PROVIDER`).
- **Recursos:** 1 CPU, 1 GB de RAM y clon enlazado (disco diferencial).
- **Aprovisionamiento:** cuatro *provisioners* Bash (`sistema`, `node`, `servidor`, `login-info`).

Dónde está: `parte1-vagrant/Vagrantfile`.

Cómo se comprueba: `vagrant up` termina sin errores y `vagrant status` indica `running`. Probado en tu PC.

Estado: ✅

R1.2. Servidor Node.js accesible desde la VM y desde el exterior

Enunciado (pág. 1): «…donde se ejecute un servidor web Node.js que debe ser accesible tanto desde la propia máquina virtual, cómo desde el exterior.»

Qué hay que implementar:

- el servidor escuchando en la VM;
- una vía de entrada desde fuera de la VM.

Cómo se ha implementado:

- **El servidor:** servicio systemd `ssr-server` que ejecuta `node app.js` y escucha en todas las IPs de la VM.
- **La entrada desde fuera:** redirección de puertos de VirtualBox, sin `host_ip`, así que escucha en todas las IPs del anfitrión.
- **Cortafuegos:** `ufw` permite el puerto 3000.

Dónde está: `Vagrantfile`: `SCRIPT_SERVIDOR`, `forwarded_port` y `SCRIPT_SISTEMA` (ufw).

Cómo se comprueba:

- `vagrant ssh -c "curl -s localhost:3000"`;
- `curl http://localhost:8080` desde el anfitrión;
- `curl http://<IP-del-anfitrión>:8080` desde otro equipo.

Estado: ✅

R1.3. Instalar Node, descargar el servidor de GitHub y ejecutarlo

Enunciado (pág. 1): «…el script que desarrolle deberá contener todas las sentencias necesarias para instalar el framework de Node, descargar de GitHub (o cualquier otro repositorio Git) el código del servidor y ejecutarlo.»

Qué hay que implementar: las tres acciones dentro del script, no a mano.

Cómo se ha implementado:

- **Instalar Node:** `SCRIPT_NODE` instala Node.js 22 desde NodeSource. Solo lo hace si no está, así que es idempotente.
- **Descargar el servidor:** `SCRIPT_SERVIDOR` hace `git clone` (o `git pull` si ya existe) de `gisai/SSR-master-server` en `/opt/ssr-master-server` y ejecuta `npm install --omit=dev`.
- **Ejecutarlo:** crea y activa `ssr-server.service`.
- **Fallo del servidor original:** `npm start` falla con `EADDRINUSE` por el doble `listen(3000)` de `app.js` y `bin/www`. Por eso el servicio ejecuta `node app.js`.

Dónde está: `Vagrantfile`.

Cómo se comprueba: `systemctl status ssr-server` → `active (running)`.

Estado: ✅

R1.4. Usar SSR-master-server (o uno propio)

Enunciado (pág. 1): «Como servidor web puede emplear el disponible en el siguiente enlace… https://github.com/gisai/SSR-master-server»

Cómo se ha implementado: se usa ese repositorio sin modificar.

Estado: ✅

R1.5. Redirección de puertos 8080 → 3000

Enunciado (pág. 1): «Debe aplicar redirección de puertos, para que el servidor sea accesible desde el exterior en el puerto 8080, pero se accesible desde la propia máquina virtual en el puerto 3000.»

Cómo se ha implementado:

```
        config.vm.network "forwarded_port", guest: 3000, host: 8080, protocol: "tcp"
```

Cómo se comprueba: `curl http://localhost:8080` (anfitrión) y `curl http://localhost:3000` (VM) devuelven el mismo HTML.

Estado: ✅

R1.6. Dirección de red acorde al tipo de conexión

Enunciado (pág. 1): «Asegúrese también de asignar a la máquina virtual una dirección de red, acorde al tipo de conexión que haya escogido (pública o privada).»

Cómo se ha implementado:

- **Red privada (host-only):** 192.168.56.10/24, además de una segunda red privada 192.168.57.10/24.
- **Red pública (opcional):** en modo puente, con IP por DHCP de la red física, activable con `BRIDGE_IF=<interfaz>`.

Cómo se comprueba: `curl http://192.168.56.10:3000`.

Estado: ✅

R1.7. Entregar el script Ruby

Enunciado (pág. 1): «Como resultado de la práctica se deberá entregar el correspondiente escrito en Ruby (Vagrant).»

Dónde está: `parte1-vagrant/Vagrantfile`.

Estado: ✅

R1.8. Configuraciones adicionales y esquemas de red avanzados

Enunciado (pág. 1): «Se valorará también que los alumnos exploren configuraciones adicionales a las mínimas requeridas, y que experimenten con esquemas de red avanzados (como por ejemplo que una misma máquina virtual disponga de diversas interfaces).»

Cómo se ha implementado:

Configuración	Detalle
Varias interfaces	NAT + 2 host-only + bridge opcional
Cortafuegos	`ufw`: solo entra lo dirigido a los puertos 22 y 3000
Seguridad del servicio	Corre con el usuario sin privilegios `nodeapp`
Teclado y locale	Teclado español, `es_ES.UTF-8`, zona horaria Europe/Madrid (petición propia)
Información al iniciar sesión	Al entrar se imprime la configuración del servidor (petición propia)
Recursos	Tope de CPU del 80 % y clon enlazado
Convivencia con KVM	*Trigger* que descarga KVM y carga VirtualBox antes de `vagrant up`

Estado: ✅

R1.9. Estudio de consumo de memoria y CPU

Enunciado (pág. 1): «Estudie el consumo en memoria y CPU de este servicio virtualizado…»

Cómo se ha implementado: medidas en tu PC (Ryzen 7 7700X, 30 GiB) en reposo y con 25 000 peticiones a `/record`, con `ps`, `VBoxManage metrics`, `free`, `top` y `du`.

Recurso	En reposo	Con carga
RAM en el anfitrión	~1.04 GiB	—
RAM usada dentro de la VM	~200 MiB	—
CPU del anfitrión	0.06 %	2.3 %
Peticiones atendidas	—	122/s

Dónde está: `doc/development/parte1/consumo_raw.txt` y `memoria_parte1.pdf` §4.1–4.3.

Estado: ✅

R1.10. Consumo de una red que replique el laboratorio

Enunciado (pág. 1): «…y discuta cuánto consumiría el despliegue de una red virtualizada que replicase la infraestructura del laboratorio (cuyo dibujo encuentra en Moodle).»

Cómo se ha implementado:

- **Escenarios:** a partir del plano, réplica completa (38 VMs) y reducida (9 VMs).
- **Variantes:** con 1 GB por VM y con asignación ajustada.

- **Contraste con datos reales:** la red de la parte 2 (9 VMs con KVM, 1 GB asignado a cada una) ocupa 6.1 GiB de RAM y unos 4.5 GB de disco.

Dónde está: `memoria_parte1.pdf` §4.4 y `red_laboratorio_3103.pdf` §5.

Estado: ✅

R1.11. Vigilar con `watch` cómo crecen las imágenes

Enunciado (pág. 1): «Puede usar el comando watch de Linux para vigilar la manera en que las imágenes crecen según continúa la operativa de la red.»

Cómo se ha implementado: se midió el tamaño de la imagen antes y después de la carga, con `ls -alh` y `du`. No creció de forma visible: los datos generados son de unos 0.4 MiB.

Estado: ⚠️ No hay una captura de `watch` durante la carga. Es opcional («puede usar»). Si se quiere, basta con ejecutarlo mientras se repite la carga:

```
        watch "ls -al ~/VirtualBox\ VMs/agr-nodeserver/Snapshots/"
```

R1.12. Comparar con servidores comerciales

Enunciado (pág. 2): «Se valorará positivamente que compare los resultados obtenidos con la capacidad de los servidores comerciales de la actualidad.»

Cómo se ha implementado: comparación con un servidor de 2 sockets con 1 TB de RAM.

Cálculo	Resultado
VMs como la de la parte 1	~860
Réplicas completas del laboratorio	~22 sin optimizar, ~70 optimizadas
Comparación con contenedores	Se discute en la memoria

Dónde está: `memoria_parte1.pdf` §4.5.

Estado: ✅

R1.13. Documento opcional de pruebas y configuraciones

Enunciado (pág. 2): «Si lo desea puede entregar también un documento escrito donde explique las pruebas realizadas y las configuraciones que proporciona mediante su script.»

Dónde está:

- `memoria_parte1.pdf`;
- `manual_usuario.pdf`;
- `guia_paso_a_paso.pdf` (parte 1).

Estado: ✅

PARTE 2 — Virtualización de infraestructura de red con KVM (págs. 3–5)

R2.1. Script que despliegue la red del laboratorio 3103 con QEMU-KVM

Enunciado (pág. 3): «Se pide desarrollar un script que despliegue una red virtualizada, según el dibujo mostrado a continuación (refleja la infraestructura montada en el laboratorio 3103) y empleando la tecnología QEMU-KVM»

Qué hay que implementar: un programa que, sin intervención manual, cree todas las máquinas y las redes del plano usando KVM.

Cómo se ha implementado:

- **La red, montada a mano con QEMU-KVM** (pasos 0–11 de la guía):
  - discos qcow2 encadenados (`base ← molde ← VM`);
  - 9 bridges;
  - 9 XML;
  - `virsh define/start`;
  - configuración interna de cada VM.
- **Parte ya automatizada:** `parte2/red.sh` prepara el anfitrión:
  - carga KVM;
  - crea los 9 bridges;
  - quita el filtrado del cortafuegos sobre los bridges;
  - da al anfitrión su IP y sus rutas.

Dónde está: `parte2/` y `guia_paso_a_paso.pdf`.

Cómo se comprueba: `sudo virsh list --all` → las 9 VMs `running`.

Estado: ⚠️ La red está desplegada con QEMU-KVM y verificada, pero el despliegue de las VMs (discos, XML, `define`/`start`, configuración interna) aún no está en un script. **Pendiente:** escribir el script que automatice los pasos de la guía.

R2.2. Lenguaje del script

Enunciado (pág. 3): «Puede emplear el lenguaje que se prefiera, aunque aconsejamos el uso de Python y Bash…»

Cómo se ha implementado:

- **Bash:** `red.sh` y `molde/preparar_molde.sh`.
- **Python:** el script de despliegue pendiente usará Python con `lxml` (ver R2.4).

Estado: ✅

R2.3. Generar los ficheros XML de todas las VMs

Enunciado (pág. 3): «El script correspondiente deberá generar los ficheros XML necesarios para desplegar todas las máquinas virtuales.»

Qué hay que implementar: un XML de libvirt por VM, generado a partir de la plantilla.

Cómo se ha implementado: cada XML sale de la plantilla de Moodle con tres sustituciones `sed`:

- `<name>XXX</name>`;
- la ruta del disco `/mnt/tmp/XXX/XXX.qcow2`;
- `<source bridge="XXX"/>`.

Además, se añade un bloque `<interface>` por cada red adicional, en orden (`eth0`, `eth1`, `eth2`). Ejemplo, RD:

```
        sed -e 's|<name>XXX</name>|<name>rd</name>|' \
            -e 's|/mnt/tmp/XXX/XXX.qcow2|/var/lib/libvirt/images/lab3103/rd.qcow2|' \
            -e 's|<source bridge="XXX"/>|<source bridge="br-bd"/>|' \
            -e 's|</interface>|</interface>\n<interface type="bridge">...br-
red3...\n...br-red4...</interface>|' \
            plantilla.xml > xml/rd.xml
```

Los 10 XML (9 VMs + molde) validan con `virt-xml-validate`.

Dónde está: `parte2/plantilla.xml` y `parte2/xml/*.xml`.

Estado: ⚠️ Los XML están generados a partir de la plantilla, con órdenes y no a mano, pero todavía no los genera el script de despliegue.

R2.4. Uso de una librería XML (lxml)

Enunciado (pág. 3): «…se utilizará alguna de las librerías disponibles en Python para leer y modificar ficheros XML. Se recomienda utilizar la librería lxml.etree…»

Cómo se ha implementado: los XML actuales se generan con `sed`.

Estado: ❌ Pendiente. El script de despliegue hará los mismos cambios con `lxml.etree`: `parse` de la plantilla, `find("name").text`, `find("devices/disk/source").set("file", …)` y un `SubElement("interface")` por bridge.

R2.5. Las cuatro subredes inferiores pueden ser un único PC

Enunciado (pág. 3): «…las cuatro subredes reflejadas en el dibujo pueden consistir en un único PC (qué puede ser incluso el anfitrión si se desea).»

Cómo se ha implementado: un PC por red (PC1–PC4, 10.0.N.10/24, puerta de enlace 10.0.N.1).

Estado: ✅

R2.6. Desplegar todas las máquinas

Enunciado (pág. 4): «El script deberá no solamente desplegar todas las máquinas…»

Cómo se ha implementado: 9 VMs (RA, RB, RC, RD, SRV, PC1–PC4), cada una con su disco diferencial sobre el molde y su XML. El PC Profesor es el anfitrión.

Cómo se comprueba: `sudo virsh list --all` → 9 en `running`.

Estado: ✅ (desplegadas; su automatización forma parte de R2.1).

R2.7. Instalar la *Software Routing Suite* en los routers

Enunciado (pág. 4): «…sino además instalar la Software Routing Suite en aquellas máquinas que actúen como router…»

Cómo se ha implementado:

- **Instalación:** FRRouting (FRR 7.2.1), sucesor de Quagga, se instaló en el molde con `apt install frr` y se dejó desactivado.
- **Activación:** solo en RA–RD (`systemctl enable --now frr`).
- **Administración:** con `vtysh`.

Dónde está: `parte2/molde/preparar_molde.sh` y `parte2/vm/r*/frr.conf`.

Cómo se comprueba: `systemctl is-active frr` → `active` en los routers. En la verificación final, `vtysh -c 'show ip route'` responde en los cuatro.

Estado: ✅ Si el profesor exigiera Quagga literalmente, Ubuntu 20.04 aún lo incluye y el cambio es directo.

R2.8. Rutas estáticas para conectividad global

Enunciado (pág. 4): «…y configurar las rutas estáticas correspondientes para lograr conectividad global.»

Cómo se ha implementado:

- **Rutas del plano** en `vtysh` (`ip route …`), guardadas con `write memory`:

Router	Rutas
RA	0/0 → 10.1.0.2
RB	10.0.1–2.0/24 → 10.2.0.2; 10.0.3–4.0/24 → 10.3.0.2; 0/0 → 10.1.0.1
RC	0/0 → 10.2.0.1
RD	0/0 → 10.3.0.1

- **Reenvío IP:** activado en los routers (`99-router.conf`).
- **Puertas de enlace:** PCs y SRV con `gateway` en `/etc/network/interfaces`.

Dónde está: `parte2/vm/<vm>/frr.conf`, `99-router.conf` e `interfaces`.

Cómo se comprueba:

- `show ip route` en los 4 routers, con capturas reales del 10/11/2026;

- traceroutes por los caminos del plano;
- las 9 VMs con 18/18.

Estado: ✅

R2.9. Servicio web en el servidor

Enunciado (pág. 4): «En el servidor se deberá desplegar un servicio web, para el que puede reciclar el código que empleó en el apartado anterior.»

Cómo se ha implementado: el mismo SSR-master-server y el mismo tipo de servicio systemd que en la parte 1.

- Se instaló en el molde (Node.js 22, `git clone`, `npm install`).
- Solo está activado en SRV (`systemctl enable --now ssr-server`), en 10.100.0.11:3000.

Dónde está: `parte2/molde/preparar_molde.sh` y `parte2/molde/ssr-server.service`.

Cómo se comprueba: `ss -tln | grep 3000` → `*:3000`.

Estado: ✅

R2.10. Acceso desde la parte inferior al servicio web

Enunciado (pág. 4): «El objetivo final de la infraestructura es lograr que cualquiera de las máquinas situadas en la parte inferior de la red tenga acceso al servicio web del servidor.»

Cómo se comprueba:

- PC1–PC4 obtienen `web=200` en la verificación final;
- `/record` desde PC1 y PC4 creó sus CSV en SRV.

Estado: ✅

R2.11. Acceso del anfitrión al servidor

Enunciado (pág. 4): «La máquina anfitriona también deberá tener acceso al servidor, en el caso de que ésta no forme parte ya de una de las subredes de la parte inferior.»

Cómo se ha implementado:

- **Conexión:** el anfitrión es el PC Profesor del plano, con 10.200.0.2/30 en `br-prof`, unido a RA ge0/0/5 (10.200.0.1).
- **Rutas:** a las 8 redes del laboratorio vía RA, sin tocar la salida a Internet del anfitrión.
- **Quién lo hace:** `red.sh`.

Cómo se comprueba:

- `curl http://10.100.0.11:3000/` → HTML;
- `/record` creó `profesor-…csv` y `final-…csv`;
- `traceroute` a PC4 → RA → RB → RD → PC4.

Estado: ✅

R2.12. Entregar el script

Enunciado (pág. 4): «Como resultado de la práctica, deberá entregar el script correspondiente…»

Dónde está: `parte2/`:

- `red.sh`;
- `molde/preparar_molde.sh`;
- XML;
- configuraciones de cada VM.

Estado: ⚠️ Falta el script de despliegue completo (R2.1).

R2.13. Memoria: plan de numeración con dibujo

Enunciado (pág. 4): «…además de una memoria explicando el plan de numeración que ha dispuesto para la infraestructura (haga un dibujo)…»

Dónde está:

- `red_laboratorio_3103.pdf` §1–2;
- `diagrama_red_lab3103.pdf/png`;
- `diagrama_construccion_lab3103.pdf/png`.

Estado: ✅

R2.14. Memoria: tablas de enrutamiento de los routers

Enunciado (pág. 4): «…así como las tablas de enrutamiento que ha configurado en los distintos router.»

Dónde está: `red_laboratorio_3103.pdf` §2.3 y `doc/development/parte2/verificacion_final.txt`.

Estado: ✅ Las cuatro tablas son capturas reales (10/11/2026).

R2.15. Plan de pruebas que confirme la conectividad global

Enunciado (pág. 4): «Deberá también indicar el plan de pruebas que ha llevado a cabo para confirmar que existe conectividad global.»

Cómo se ha implementado:

- **Durante la construcción:** pruebas en cada paso (ping, traceroute, `curl`).
- **Prueba final:** cada equipo hace ping a las 18 IPs del laboratorio y una petición HTTP al servidor. Primero en los 10 equipos (180/180 y 10/10); se repitió el 10/11/2026 tras reiniciar el anfitrión, con las 9 VMs en 18/18 y `web=200`.

Dónde está:

- `red_laboratorio_3103.pdf` §3–4;
- `doc/development/parte2/verificacion_final.txt`;
- `doc/development/parte2/capturas/`.

Estado: ✅

R2.16. Configuraciones adicionales en las VMs y en la Software Routing Suite

Enunciado (pág. 4): «Se valorará que los alumnos apliquen configuraciones adicionales a las mínimas requeridas en cualquiera de las máquinas virtuales, así como en el Software Routing Suite.»

Cómo se ha implementado:

- **En las VMs:**
  - imagen molde común con discos diferenciales (4.5 GB en total frente a unos 29 GB copiando la imagen entera);
  - hostname de cada equipo;
  - servicio web con usuario sin privilegios y reinicio automático;
  - comentarios con el puerto del plano en `/etc/network/interfaces`.
- **En FRR:** hostname de cada router y configuración integrada guardada con `write memory`.

Estado: ⚠️ Es poco, sobre todo en FRR. Propuestas:

- ruta *blackhole* en RA para cortar el bucle RA↔RB con destinos desconocidos;
- descripciones de interfaz;
- contraseña y banner en `vtysh`;
- rutas de respaldo (*floating static*);
- variante con OSPF.

R2.17. Usar la imagen de disco y el XML de Moodle

Enunciado (pág. 4): «…aconsejamos encarecidamente que… se emplee como base la imagen de disco y el fichero XML que se puede encontrar en Moodle.»

Cómo se ha implementado:

- `base.qcow2` es la imagen de Moodle (Ubuntu 20.04.5), intacta y en solo lectura.
- Todos los XML parten de `plantilla.xml`, la de Moodle.

Estado: ✅

R2.18. Usuario root / agr; cambiar hostname y contraseña

Enunciado (pág. 4): «En dicha imagen de disco, el superusuario es root, con password: agr. Si lo desea, puede cambiar el hostname y la contraseña.»

Cómo se ha implementado: se mantiene `root`/`agr` y se cambia el hostname de cada VM (`srv`, `ra`, …, `pc4`).

Estado: ✅

R2.19. Conectar el anfitrión a un bridge de las subredes inferiores (aconsejado)

Enunciado (pág. 4): «Aconsejamos que el PC anfitrión se conecte a uno de los bridge desde las subredes inferiores (RED 1, 2, 3 y 4), para que pueda comprobar el flujo de tráfico hacia el exterior.»

Cómo se ha implementado: se ha seguido el plano, que sitúa el PC Profesor en 10.200.0.0/30 junto a RA. El propio enunciado contempla esa opción: «…en el caso de que ésta no forme parte ya de una de las subredes de la parte inferior».

Estado: ⚠️ Es una recomendación. Para seguirla al pie de la letra:

- dar al anfitrión una IP en `br-red1` (por ejemplo 10.0.1.100/24);
- poner rutas vía 10.0.1.1.

R2.20. Mensajes con print y trazas con logging

Enunciado (pág. 4): «Los mensajes que se quiera que el programa siempre muestre en pantalla se pueden imprimir utilizando la función print/printf… para la depuración… Se puede utilizar el módulo “logging” de Python.»

Estado: ❌ Pendiente: se aplicará en el script de despliegue en Python.

R2.21. Gestionar las VMs con `virsh define|start|shutdown|destroy`

Enunciado (pág. 4): «Para la gestión de las máquinas virtuales (arranque, parada, etc.) se recomienda utilizar el comando “sudo virsh define|start|shutdown|destroy”…»

Cómo se ha implementado:

Acción	Orden
Registrar	`virsh define`
Arrancar	`virsh start`
Apagar ordenadamente	`virsh shutdown`
Apagar de golpe	`virsh destroy` (+ `undefine` para borrar)

Estado: ✅

R2.22. Terminales con `xterm -e "sudo virsh console …" &`

Enunciado (pág. 5): «Si desea arrancar terminales desde el script para las distintas máquinas virtuales, puede emplear el comando xterm… Tenga en cuenta que los comandos anteriores deben ser ejecutarlos en background…»

Cómo se ha implementado: consolas con `sudo virsh console <vm>`.

Estado: ⚠️ Es opcional («si desea»). El script de despliegue podrá abrir un `xterm` por VM en segundo plano.

R2.23. `virt-cat` / `virt-ls` para inspeccionar sin entrar

Enunciado (pág. 5): «Los comandos sudo virt-cat y sudo virt-ls pueden ayudar a listar los ficheros o volcar su contenido sin necesidad de acceder a la máquina virtual.»

Cómo se ha implementado: se usaron `virt-cat` y `virt-ls` para examinar la imagen de Moodle antes de prepararla: versión de Ubuntu, `/etc/network`, `/etc/netplan` y `/boot`.

Estado: ✅ (es opcional).

Resumen

Parte	✅	⚠️	❌
Parte 1 (13 requisitos)	12	1 (captura de `watch`, opcional)	0
Parte 2 (23 requisitos)	15	6	2 (`lxml` y `print`/`logging`: van con el script)

Pendientes:

1. **Script de despliegue** (R2.1, R2.3, R2.4, R2.12, R2.20, R2.22): en Python con `lxml`, que automatice los pasos de la guía con los ficheros de `parte2/`:
   - crear discos;
   - generar los XML;
   - escribir la configuración de cada VM;
   - `define`/`start`;
   - ejecutar las pruebas.
2. **(Opcional)** Más configuración en FRR (R2.16).
3. **(Opcional)** Anfitrión en una red inferior (R2.19) y captura de `watch` (R1.11).